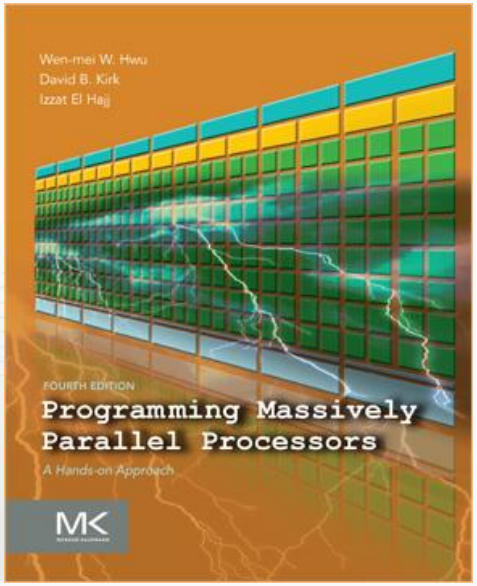
MORGAN KAUFMANN

Programming Massively Parallel Processors

A Hands-on Approach

CHAPTER 4 › Compute Architecture and Scheduling

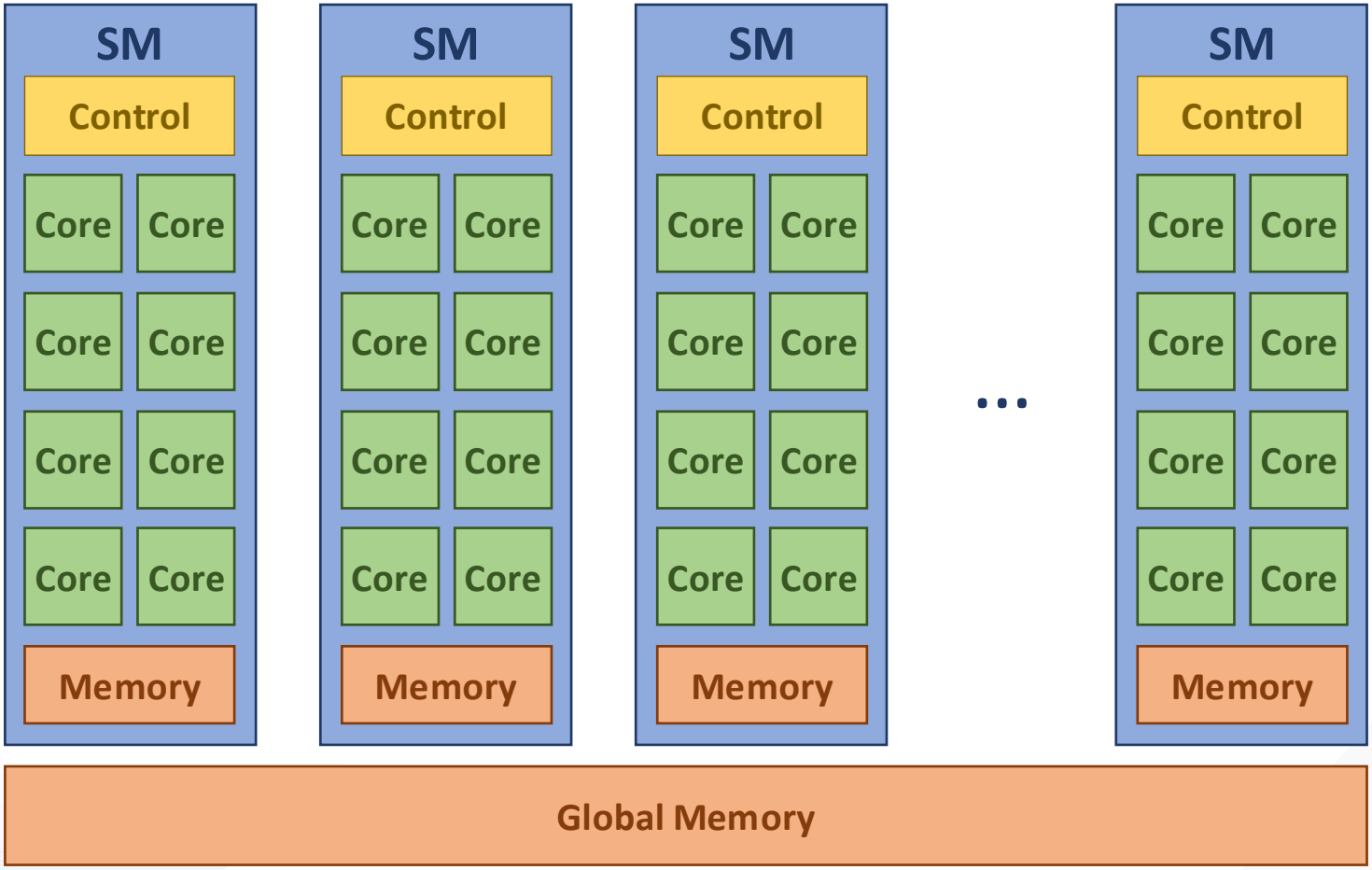


Copyright © 2022 Elsevier

GPU Architecture

A GPU consists of multiple **Streaming Multiprocessor (SMs)**, each consisting of multiple **cores** with shared **control** and **memory**

(e.g., a Volta V100 GPU has 80 SMs with 64 cores each which totals 5120 cores)

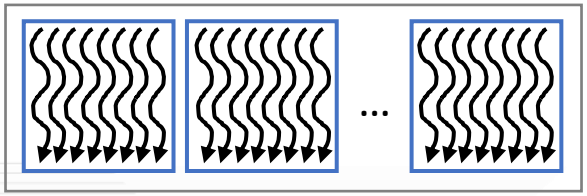


Copyright © 2022 Elsevier

Assigning Blocks to SMs

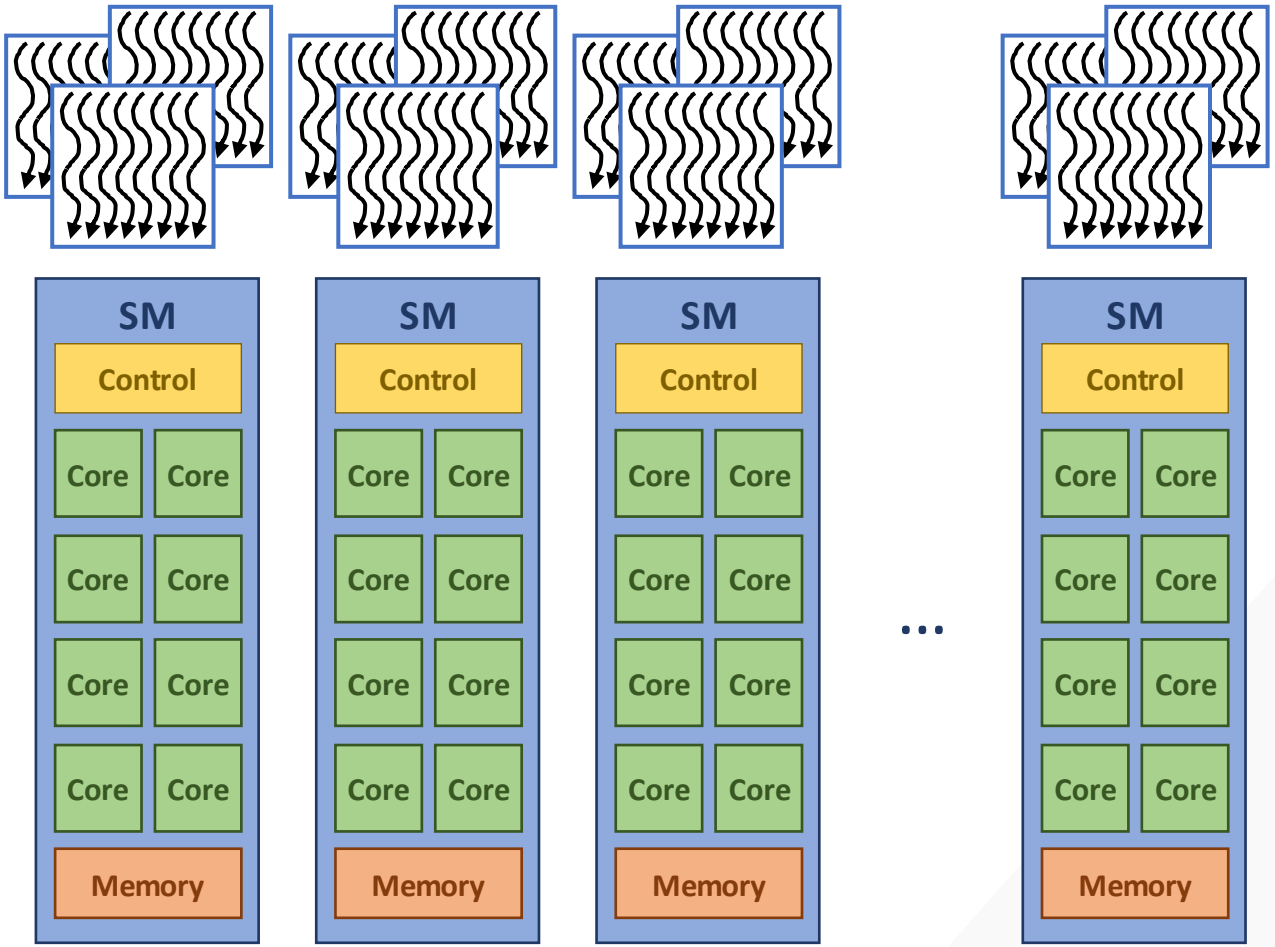
Threads/blocks require resources to execute (e.g., registers, memory, …) so SMs can accommodate a limited number of threads/blocks at once…

… The remaining blocks wait for other blocks to finish before they can be assigned to an SM.



Waiting

Threads are assigned to SMs at block granularity
(all threads in a block are assigned to the same SM)



Copyright © 2022 Elsevier

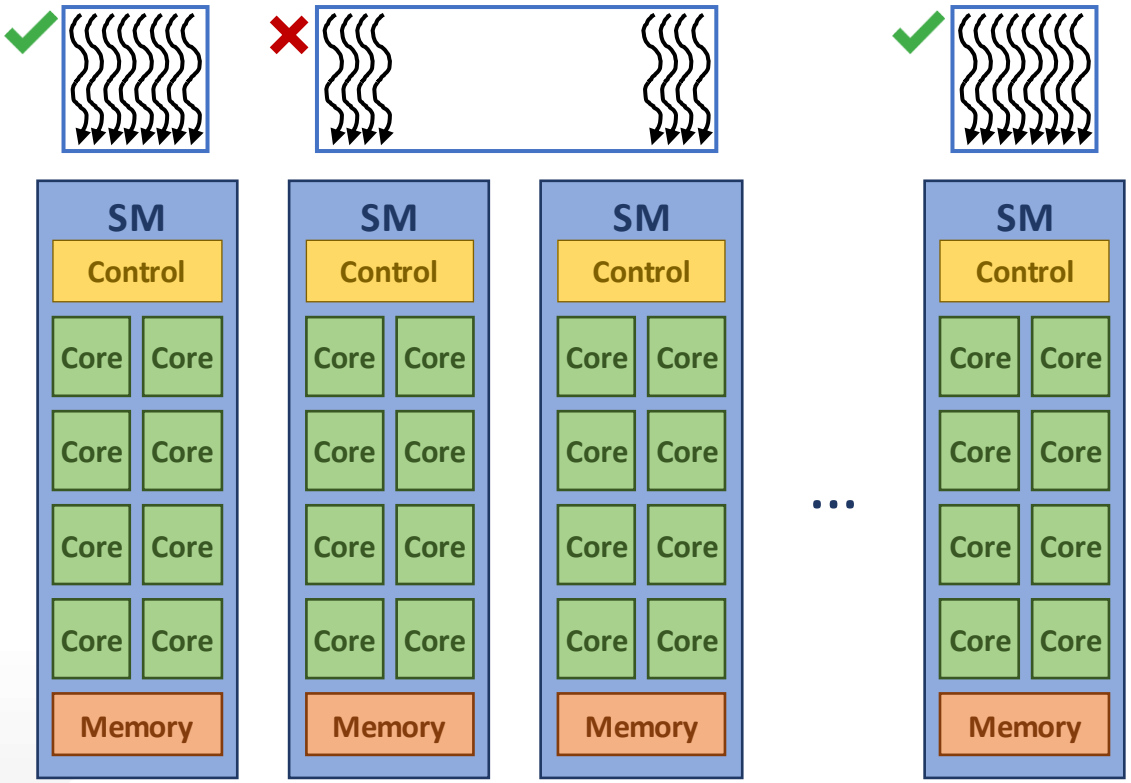
Synchronization

- Threads in the same block can collaborate in ways that threads in different blocks cannot:
  - Barrier synchronization: `__syncthreads()`
    - Wait for all threads in the block to reach the barrier before any thread can proceed
  - Shared memory (discussed later)
    - Access a fast memory that only threads in the same block can access
  - Others (discussed later)

Copyright © 2022 Elsevier

Scheduling Considerations

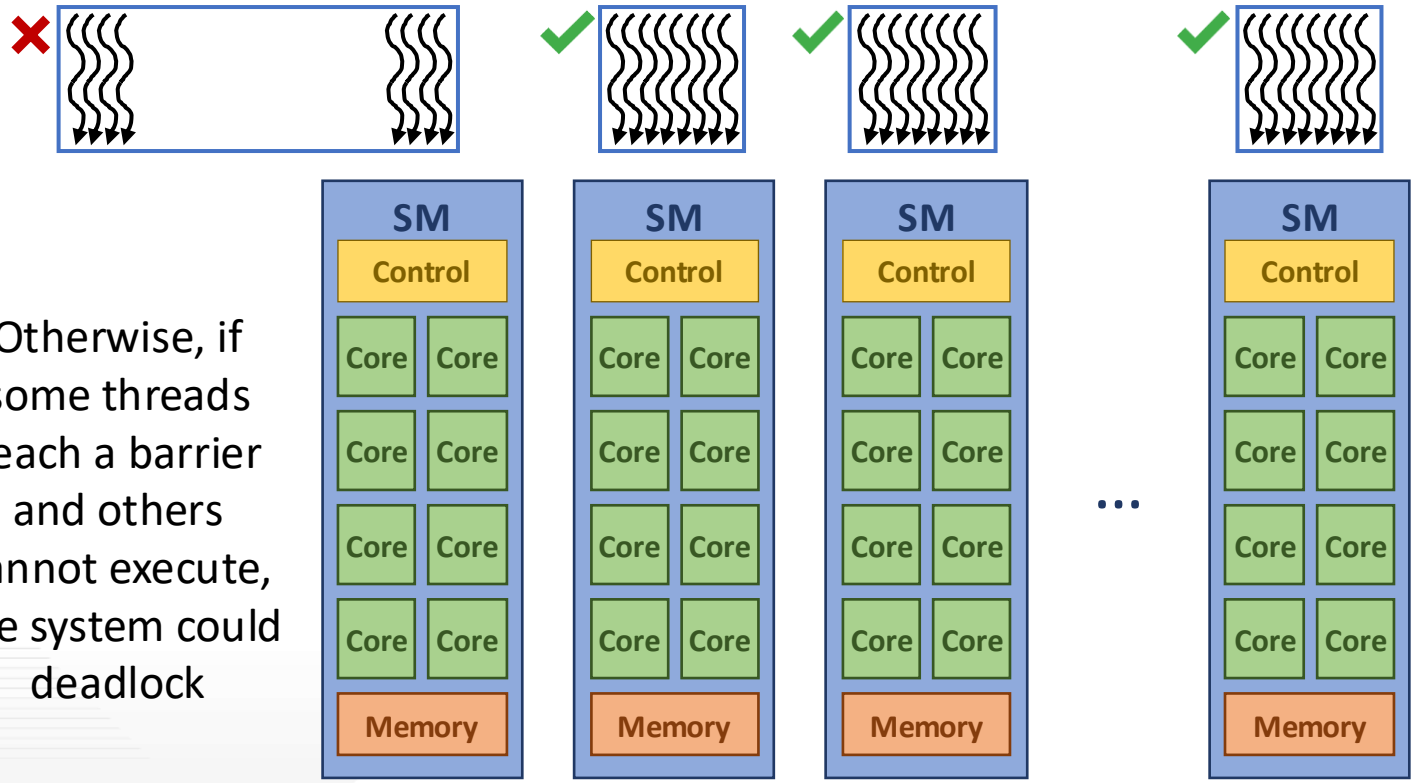
- Threads in the same block are assigned to the same SM
  - Assigning threads in the same block to the same SM makes supporting collaboration between them efficient



Copyright © 2022 Elsevier

Scheduling Considerations

- All threads in a block are assigned to an SM simultaneously
  - A block cannot be assigned to an SM until it secures enough resources for all its threads to execute



Otherwise, if some threads reach a barrier and others cannot execute, the system could deadlock

Copyright © 2022 Elsevier

Transparent Scalability

- Threads in different blocks do not synchronize *
  - Allows blocks to execute in any order
  - Allows blocks to execute both in parallel with each other or sequentially with respect to each other

- Enables **transparent scalability**
  - Same code can run on different devices with different amounts of hardware parallelism
    - Execute blocks sequentially if device has few SMs
    - Execute blocks in parallel if device has many SMs

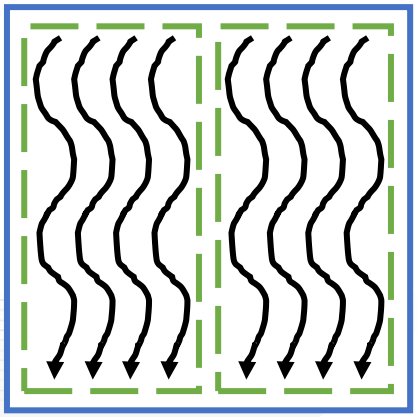
- Programmers should not write code that tries to synchronizes across blocks
  - Deadlock occurs if the blocks are not scheduled simultaneously
  - Note: ***cooperative groups*** is an advanced feature of CUDA that allows synchronization across blocks by limiting the number of blocks to guarantee that all blocks are executing simultaneously

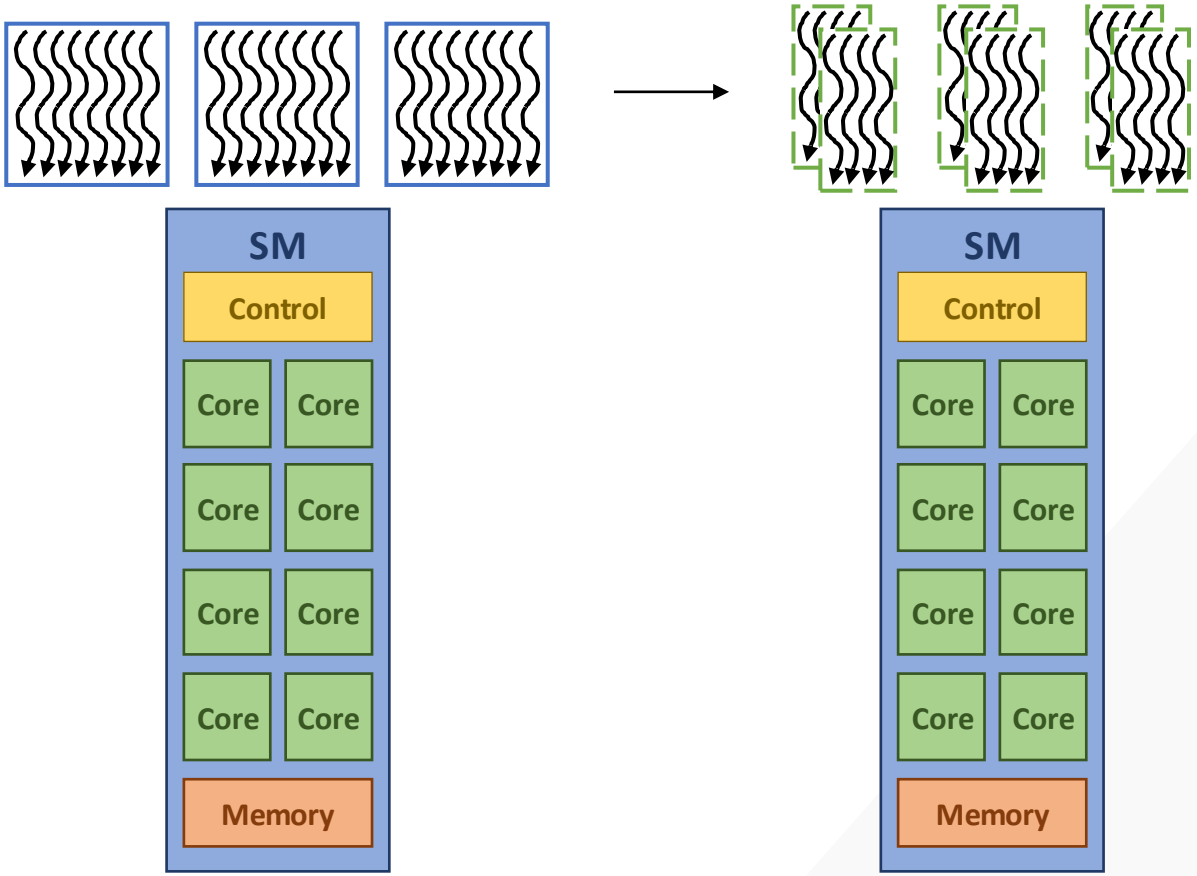
Copyright © 2022 Elsevier

Thread Scheduling

- Threads assigned to an SM run concurrently
  - The SM has a scheduler that manages their execution

Blocks assigned to an SM are further divided into **warps** which are the unit of scheduling



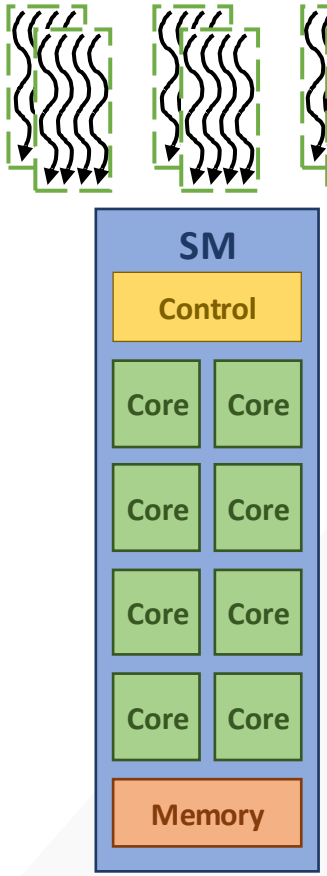


Copyright © 2022 Elsevier

Warps

- Warps are the unit of scheduling in an SM
  - The size of warps is device-specific, but has always been 32 threads to date

- Threads in a warp are scheduled together and executed following the **SIMD** model
  - Single Instruction, Multiple Data
  - One instruction is fetched and executed by all the threads in the warp, each processing different data



Copyright © 2022 Elsevier

Why SIMD?

- Advantage of SIMD:
  - Share the same instruction fetch/dispatch unit across multiple execution units (cores)
    - i.e., amortizes the cost of control across more execution units

Copyright © 2022 Elsevier

Real Stuff



Example: Volta V100 GPU

- SM has 64 FP32 cores
- Organized into 4 processing blocks
- Each processing block has 16 cores that share a dispatch unit
- A warp is executed by one processing block

Source:
NVIDIA Tesla V100 GPU Architecture, 2017

Copyright © 2022 Elsevier

Why SIMD?

- Advantage of SIMD:
  - Share the same instruction fetch/dispatch unit across multiple execution units (cores)
    - i.e., amortizes the cost of control across more execution units

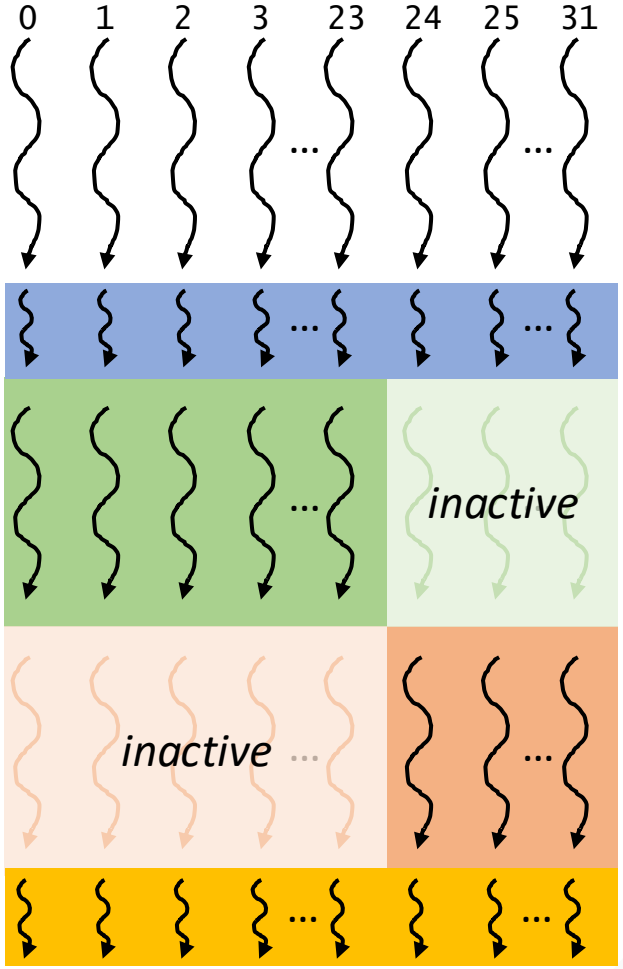
- Disadvantage of SIMD:
  - Different threads taking different execution paths result in **control divergence**
    - Warp does a pass over each unique execution path
    - In each pass, threads taking the path execute while others are disabled
  - The percentage of threads/cores enabled during SIMD execution is called the **SIMD efficiency**

Copyright © 2022 Elsevier

Control Divergence Example

Example 1: Threads take different branches of a conditional

```
if(threadIdx.x < 24) {
    A
} else {
    B
}
C
```



Copyright © 2022 Elsevier

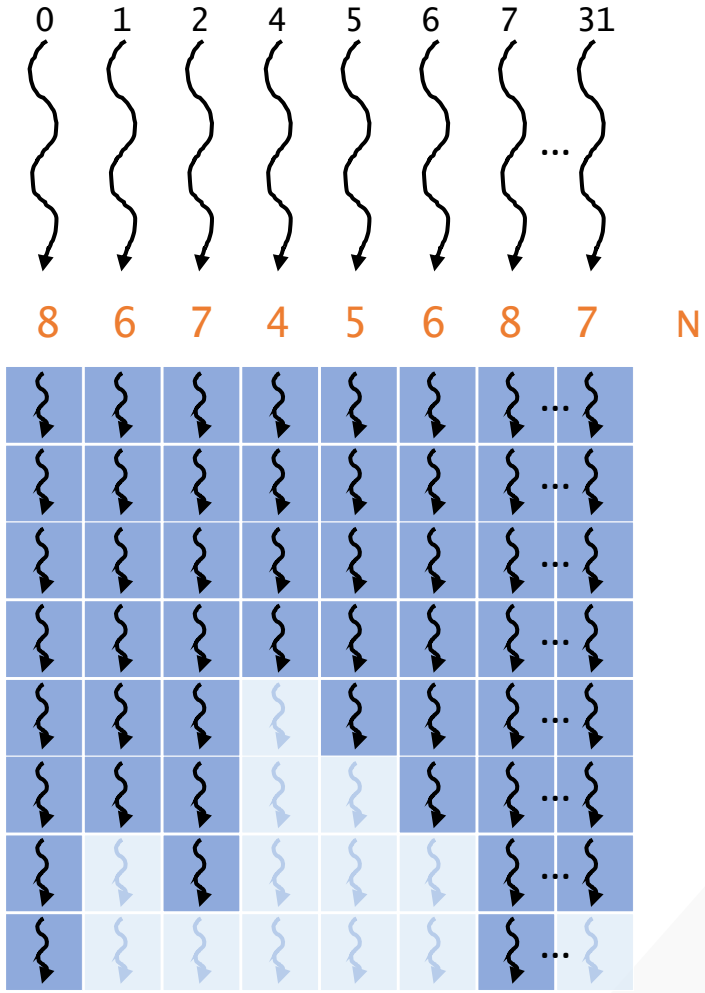
Control Divergence Example

Example 2: Threads execute a different number of loop iterations

```
N = a[threadIdx.x];
for(i = 0; i < N; ++i) {

    A

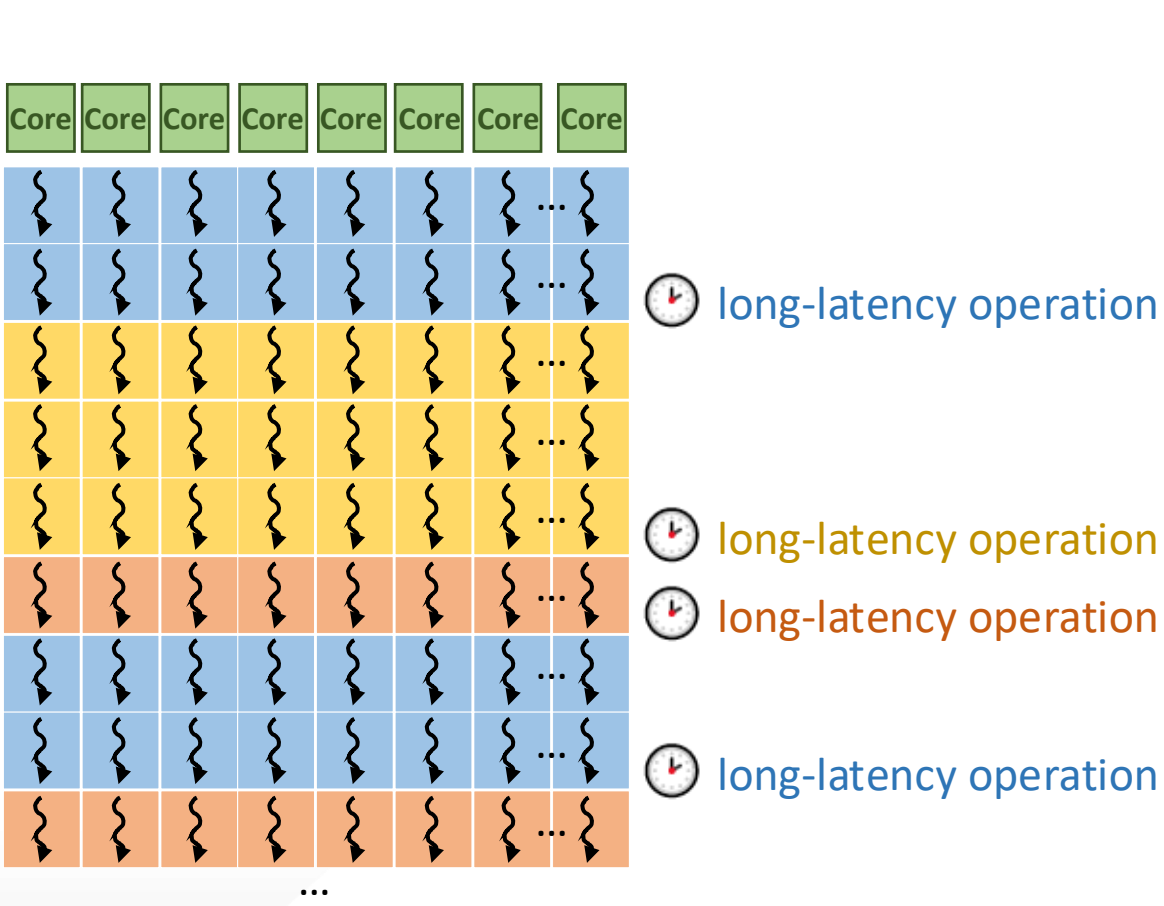
}
```

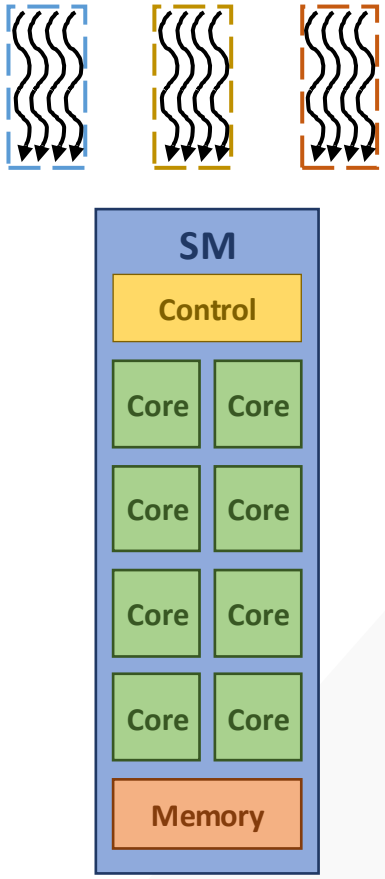


Copyright © 2022 Elsevier

Latency Hiding

- When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution





Copyright © 2022 Elsevier

Latency Hiding

- When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution

- Many warps are needed so that there is sufficient work available to hide long latency operations
  - i.e., there is high chance of finding a warp that is ready

- For this reason, an SM typically supports many more threads than the number of cores it has

Copyright © 2022 Elsevier

SM Resources

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Compute Capability	3.5	5.2	6.0	7.0
Number of SMs	15	24	56	80
Cores per SM	192	128	64	64
Max threads per SM	2048	2048	2048	2048
Max blocks per SM	16	32	32	32
Max threads per block	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB

Max threads per SM is much higher than cores per SM

Copyright © 2022 Elsevier

Occupancy

- The **occupancy** of an SM refers to the ratio of the warps or threads active on the SM to the maximum allowed

- In general, maximizing occupancy is desirable because it improves latency hiding
  - Common case, but possible to have cases where lower occupancy is desirable

Copyright © 2022 Elsevier

Occupancy Constraints

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Compute Capability	3.5	5.2	6.0	7.0
Number of SMs	15	24	56	80
Cores per SM	192	128	64	64
Max threads per SM	2048	2048	2048	2048
Max blocks per SM	16	32	32	32
Max threads per block	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB

Choice of threads per block constrains occupancy

Copyright © 2022 Elsevier

Occupancy Example

- Volta V100 supports up to: 1024 threads/block
  2048 threads/SM, and **32 blocks/SM**
  - If **256 threads/block** are used, occupancy is maximized
    - (**2048 threads**)/(**256 threads/block**) = 8 blocks < **32 blocks**
  - If **32 threads/block** are used, the occupancy will be limited by the number of blocks allowed
    - (**2048 threads**)/(**32 threads/block**) = 64 blocks > **32 blocks**
  - If **768 threads/block** are used, occupancy will be limited because 2048 is not evenly divisible by 768
    - (**2048 threads**)/(**768 threads/block**) = 2 blocks
    - The remaining 512 cannot be used

Copyright © 2022 Elsevier

Querying Available Resources

- Programmers can query the properties of a device to identify its available resources

```
cudaError_t cudaGetDeviceProperties(cudaDeviceProp* prop,
                                    int  device)
```

- Example:

```
int devID = 0;
cudaDeviceProp devProp;
cudaGetDeviceProperties(&devProp, devID);
int maxThreadsPerBlock = devProp.maxThreadsPerBlock;
int maxThreadsPerMultiProcessor =
                devProp.maxThreadsPerMultiProcessor;
```

Copyright © 2022 Elsevier

References

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.

Copyright © 2022 Elsevier